

一种差分演化 Q 表的改进 Q-Learning 方法^{*}

李 骁, 曹子建, 贾浩文, 郭瑞麒

(西安工业大学 计算机科学与工程学院, 西安 710021)

摘 要: 针对 Q-Learning 算法在路径搜索应用中的盲目性而导致收敛速度慢、回报效率低的问题,文中提出了一种差分演化 Q 表的改进 Q-Learning 方法(DE-Q-Learning)。改进算法利用差分演化算法的全局搜索优势,将由 Q 表个体组成的演化种群通过变异、交叉和选择操作选择出较好的初始 Q 表,以此提升 Q-Learning 前期回报与探索能力。文中在 OpenAI 的 Gym 环境中验证了 DE-Q-Learning 方法的有效性,并进一步在复杂迷宫环境和强化学习环境 Pacman 中实验了其在复杂路径搜索和动态避障问题上的性能。实验结果表明,DE-Q-Learning 在 Pacman 环境中相比于改进算法 Double-Q-Learning 和 SA-Q-Learning 不仅在历史回报方面具有明显优势,而且收敛速度分别提升了 42.16% 和 15.88%,这表明 DE-Q-Learning 能够显著提高历史累积回报和算法的收敛速度。

关键词: 强化学习;差分演化;Q-Learning;Q 表

中图分类号: TP273

文献标志码: A

文章编号: 1673-9965(2023)04-0369-014

Improved Q-Learning Method with Differential Evolution by Optimizing Q-Table

LI Xiao, CAO Zijian, JIA Haowen, GUO Ruiqi

(School of Computer Science and Engineering, Xi'an Technological University, Xi'an 710021, China)

Abstract: In response to the slow convergence and low return resulting from the limitations of Q-Learning algorithms when applied in path search, this paper proposes a path search method called Differential Evolution Q-Learning (DE-Q-Learning) that improves Q-Learning by optimizing Q-table. The proposed algorithm leverages the global optimality of the differential evolution algorithm, selecting an evolving population composed of Q-table individuals with excellent initial values through mutation, crossover, and selection operations to enhance the Q-Learning payoff and exploration ability. Firstly, the effectiveness of the DE-Q-Learning method was verified in the Open AI's Gym environment. And then its performance in complex path search and dynamic obstacle avoidance was further tested in a complex

^{*} 收稿日期:2023-03-15

基金资助:基础加强计划技术领域基金项目(2021-JCJQ-JJ0513)。

第一作者简介:李 骁(1998—),男,西安工业大学硕士研究生。

通信作者简介:曹子建(1978—),男,西安工业大学副教授,主要研究方向为演化算法、机器学习、网络安全,E-mail:bosscao@163.com。

引文格式:李骁,曹子建,贾浩文,等.一种差分演化 Q 表的改进 Q-Learning 方法[J].西安工业大学学报,2023,43(4):369-382.

LI Xiao, CAO Zijian, JIA Haowen, et al. Improved Q-Learning Method with Differential Evolution by Optimizing Q-Table [J]. Journal of Xi'an Technological University, 2023, 43(4): 369-382.

maze environment and in the reinforcement learning environment of Pacman. The experimental results show that DE-Q-Learning has notable advantages in historical returns over the improved algorithms (Double-Q-Learning and SA-Q-Learning) in the Pacman environment, with its convergence speed increased by 42.16% and 15.88%, respectively. These findings indicate that DE-Q-Learning can effectively enhance the historical cumulative return and accelerate the convergence speed.

Key words: reinforcement learning; differential evolution; Q-Learning; Q-Table

路径搜索是人工智能领域中的一个重要问题,涉及到诸如自动规划、运动控制、资源分配、游戏策略等诸多应用场景。在路径搜索问题中,通常需要在状态空间中找到一条最优路径,以满足某些约束条件或优化目标。针对不同的问题,可以采用多种搜索算法进行求解,

目前已有的路径搜索算法有很多,例如人工势场方法^[1]、蚁群算法^[2]等,但这些传统搜索算法在实际应用中面临着挑战和限制。而强化学习类型的方法能够自主探索未知环境模型,以提高路径搜索问题的求解效率和精度,故该方向成为了近年来的研究热点。

强化学习(Reinforcement Learning, RL)作为人工智能领域的一个重要分支,已经得到广大学者的高度认可,在解决序贯决策问题上,强化学习采用马尔可夫决策过程的方式,智能体通过与环境交互得到经验集于当前状态进行决策^[3]。而Q-Learning作为一种经典的无模型强化学习方法^[4]非常适合求解有限空间的路径搜索问题,其收敛能力主要取决于智能体对环境信息的探索与利用。而在优化Q-Learning探索能力方面,不同研究人员提出了多种改进方法。文献[5]引入了探索因子和深度学习因子来提高算法的探索效率,但忽略了算法的可拓展性和稳定性;文献[6]通过势场优化Q表初值,并采用多步长策略和动态调节贪婪因子来平衡探索与利用,但未考虑动态障碍物的避障策略;文献[7]提出了使用平均奖赏和相对值函数的值迭代方式来加速收敛,但计算复杂度较高且对奖励信号不敏感;文献[8]将资格迹概念融入Q-Learning中,提出了 $Q(\lambda)$ -Learning算法,减少了Q值的计算量,但缺乏实验验证;文献[9]提出了Double Q-Learning算法,通过使用两个估计器分离最优动作和最大Q值的选取,改善了收敛速度;同样,Speedy Q-Learning (SQL)和Double Speedy Q-Learning也对Q值的更新方式进行了改进,提高了收敛能力^[10-11],但这些方法通常需要更

长的训练时间;除此之外,许多学者考虑Q-Learning与演化算法结合优化的模式,将演化算法的思想融入Q-Learning中,并从多个角度对算法存在的问题进行了优化。文献[12]提出了一种结合退火算法(Simulated Annealing)中的Metropolis准则的Q-Learning改进算法(SA-Q-Learning),利用模拟退火的思想对 ϵ 的取值进行线性控制,从而保证算法更加快速找到最优策略,但并未发挥出演化算法的优势。文献[13]为了解决不确定性的POMDP近似最优解问题,在基于SA-Q-Learning算法的基础上提出了一种有限步历史信息与状态信度概率分布相结合,提出了一种直接进化策略的新算法(MA-Q-Learning),该算法提高了算法的收敛速度,但存在Q值过估计的问题。在实际工程应用中,文献[14]针对虚拟作业中存在的 uncertainty 问题,提出了一种遗传算法(Genetic Algorithm, GA)结合Q-Learning算法进行最优策略选择的策略规划模型(GA-Q-Learning),该模型有效地避免了早期较大的Q值动作,但该模型鲁棒性较差,无法有效地解决行为空间较少的问题。

上述改进算法能够提高Q-Learning算法的性能,但均未考虑Q-Learning算法在初始化时的盲目性,这会导致Q-Learning算法在前期路径搜索中长时间地无效探索,影响算法的收敛速度和平均回报。基于此,文中提出了一种利用差分演化(Differential Evolution, DE)算法对Q表进行优化来改进Q-Learning收敛能力的方法DE-Q-Learning。该方法利用演化算法优化Q表的初始化,解决了Q-Learning算法在初始环境中的盲目性问题。通过使用差分演化算法推动Q表的进化,DE-Q-Learning在时间和空间上提高了算法的探索效率。最后在复杂迷宫环境、实验环境Pacman游戏和OpenAI的gym工具包中MountainCar上进行了仿真实验,实验结果显示DE-Q-Learning算法相比于Q-Learning及其改进算法具有优异的性能,并且能够在更短的时间内快速收敛。

1 基础理论

1.1 Q-Learning

Q-Learning 是一种基于时序差分的无模型学习方法,其 Q-Learning 的核心是 Q 表, Q 表的行和列分别表示 State 和 Action 的值,用 Q 表的值 $Q(s,a)$ 来衡量当前状态采取动作 a 的价值,所以 Q-Learning 只考虑当前状态 s' 以及当前状态选择的行为 a' ,使 $Q(s',a')$ 值最大即可,智能体采用 Q-Learning 算法与环境交互如图 1 所示。

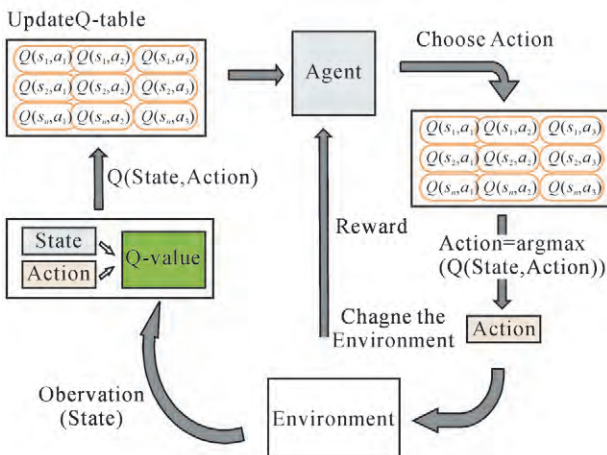


图 1 Q-Learning 算法与环境交互过程

Fig. 1 Interaction between Q-Learning algorithm and environment

通过这种局部映射的方式,无需充分地计算全局信息,只需通过局部选择的方式便可获得全局最优动作序列,进而使智能体获取最大目标收益。Q-Learning 的算法伪代码如算法 1 所示。

算法 1 Q-Learning 伪代码

初始化环境 E , 状态空间 S , 动作空间 A , 折扣因子 γ , Q 表 Q -table

初始化行为值函数 $Q(s',a') = 0, \pi(a|s) = \frac{1}{A}$

For $k = 0, 1, \dots, m$ do # agent 的每条完整路径
初始化 状态 s

For $t = 0, 1, 2, 3, \dots$, do # agent 的每一个训练步

通过贪心策略 ϵ of π 在环境 E 中采取动作 A
 $r, s' = \text{Step}(A)$ # 执行下一个动作 A 产生奖励 r 和下一个状态 s'

$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_a Q(s',a') - Q(s,a)]$ # 更新 Q 表 Q -table
 $s \leftarrow s'$

end for s is terminated

end for

$\pi^*(s) = \arg\max_a Q(s,a)$ # 根据 Q 表更新策略

Q-Learning 的状态行为值函数更新为

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_a Q(s',a') - Q(s,a)], \quad (1)$$

式中: s 为当前状态; a 为当前状态 s 所采取的动作; α 为学习率; r 为在当前状态 s 下执行动作所得到的立即回报。 γ 为折扣因子, 用来控制将来累积回报对当前状态所产生的影响。 s' 表示当前状态 s 采取动作到达的后续状态, a' 为在状态 s' 下选择的动作, 然后用新的 Q 值去更新 Q 表。

Q-Learning 分别采取两种不同的策略来进行策略评估和更新。在策略更新上 Q-Learning 采用 ϵ 贪心算法来选择动作, 在更新 Q 表时则采用贪心算法, 通过下一个状态的最大 Q 值来更新当前状态的值。而在动作选择中, Q-Learning 采用 ϵ 贪心算法, 其主要思想是在探索的同时也能同时进行一定的利用。在行为选择时采用的 ϵ -贪心算法, 其中 ϵ 值越大则探索的效果越好。反之, 若 ϵ 值越小则更加注重利用。但正因为其 ϵ 取值的固定, 往往不能更好地平衡其探索和利用的效率, ϵ 取值的不合理同时也会使得算法的搜索效率大幅度下降, 在实际案例中 ϵ 一般在 $[0, 0.1]$ 区间内取值。

1.2 差分演化算法

差分演化 (DE) 算法自 1996 年由文献 [15] 提出后, 在解决优化问题上有着广泛的适用性。DE 算法在遗传算法的启发下提出, 其思想与传统演化算法保持一致, 主要由变异、交叉、选择这三个步骤所组成, 在解决 NP (Nondeterministic Polynomial) 难问题上有着良好的适用性。对于求解以下形式的最小化问题。

$$\begin{aligned} \min f(x) \\ \text{s. t. } x \in \Omega \in R^D, \end{aligned} \quad (2)$$

式中: D 为目标解空间的维数; $\Omega \in R^D$ 则为可行解集。DE 算法关键步骤为

Step1: 变异

$$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3}). \quad (3)$$

对于变异算子的设定, 通过在 t 代随机取样三个不同个体 x_{r1}, x_{r2}, x_{r3} 来执行变异操作; F 作为变异因子, 用来调控变异程度, v_i 代表第 t 代生成的变异个体。

Step2:交叉

$$u_{i,j}^{t+1} = \begin{cases} v_{i,j}^t, & \text{if } A \cup B \\ x_{i,j}^t, & \text{otherwise} \end{cases}, \quad (4)$$

其中,令 A 为条件“ $\text{rand}(j) \leq \text{CR}$ ”, B 为条件“ $j = \text{random}(i)$ ”,且 $j = 1, 2, \dots, D$,其中 $\text{rand}(j)$ 表示生成一个在 $[0, j)$ 范围内的随机整数或随机浮点数,而 $\text{random}(i)$ 表示生成一个在 $[0, i)$ 范围内的随机浮点数, CR 为交叉因子, $u_{i,j}^{t+1}$ 代表第 t 代经过交叉操作生成的个体。

Step3:选择

$$x_i^{t+1} = \begin{cases} u_{i,j}^{t+1} & \text{if } (f(u_{i,j}^{t+1}) < f(x_{i,j}^t)) \\ x_i^t & \text{if } (f(u_{i,j}^{t+1}) > f(x_{i,j}^t)) \end{cases}. \quad (5)$$

通过选择操作将适应度值较好的个体进行保留,并作为下一代种群的初始值。

2 演化优化 Q 表的改进 Q-Learning 方法

马尔可夫模型作为强化学习与演化算法共同的基础,其定义如下:通过创建一个五元组 $\langle S, A, P, R, \gamma \rangle$ 并结合 Bellman 方程构成一个基本的马尔可夫决策过程(MDP)问题模型,其中五元组中的元素分别表示为: S (有限的状态空间), A (有限的动作空间), P (状态-动作转移概率矩阵), R (奖励函数), γ (折扣因子)。MDP 动态过程如图 2 所示,智能体初始状态为 S_0 ,从合法状态集 A 中挑选一个动作执行,得到回报 r_0 ,然后按照一定概率随机转移到下一个 S_1 状态,如此往复直到到达终止状态。在这个过程中不同的状态动作影响着最优策略的搜索进度,因此,可以使用演化算法来优化存储状态动作对的 Q 表,以加速 Q-Learning 寻找最优策略的效率。演化算法由于其固有的并行性,能够不受问题性质的限制,有效地解决传统优化问题难以处理的复杂问题,加快了策略搜索速度。从而能够有效缓解 Q-Learning 在环境信息匮乏的情况下的盲目探索性问题。

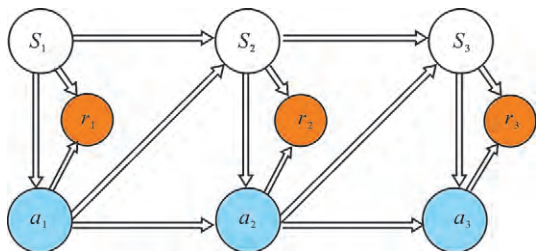


图 2 MDP 结构

Fig. 2 The structure of MDP

2.1 时间复杂度分析

Q-Learning 作为一种无模型算法,样本复杂性 T 决定了其时间复杂度 $O(T)$,空间复杂度 $O(\text{SAH})$ 更多取决于状态和动作的空间大小。其中 S 代表状态空间大小, A 代表动作空间大小, H 则表示为每一次执行所走的步长^[16]。而在解决无模型问题上,Q-Learning 在已知状态空间和动作空间的前提下,如何有效地利用环境信息是提升算法收敛能力的关键。随着模型环境复杂度的提升,不仅状态、动作对的数量增加,而且智能体在每次探索时所需的步长 H 也会随之增加。由此可以看出,环境信息对算法的运行效率起着至关重要的作用。

演化算法作为一种解决复杂问题的有效手段,可以用于解决强化学习在算法初期的盲目探索性问题。DE 具有较低的时间复杂度 $O(\text{NP} \cdot D \cdot \text{Gmax})$ ^[17-20],其中 NP 代表种群大小, D 表示维度, Gmax 为算法最大运行代数。对于复杂无模型问题,若将状态行为值函数 Q 作为 NP 的输入, S 作为 D 的输入,步长 H 则在搜索空间进行了提升。每当演化算法生成一代 Q 表种群作为 NP 输入时,步长 H 在空间上则扩展为 $\text{NP} \cdot H$,由此便提高了算法的执行效率。由于 DE 算法具有控制参数少、收敛速度快、寻优精度高及鲁棒性强的优点,因此文中选用 DE 作为演化算法实例。当然任何其它演化算法都可以作为实例算法来对 Q-Learning 进行优化,文中主要侧重于 DE-Q-Learning 算法的原理。

2.2 算法的局限性

在 Q-Learning 算法运行初期,Q 表参数的初始化会导致采集到的数据往往是未经训练的样本集。如何更加有效地提升算法的执行效率,更多程度取决于算法前期的探索能力。另外,在算法前期的探索阶段,所获取到的立即回报值不一定对算法训练有效,能够正确引导智能体进行决策的立即汇报过少,无法有效地帮助算法在前期快速收敛。同时,对于大多数问题模型而言,若属于密集回报型问题,则对算法的训练相对较好。但遇到稀疏回报问题时,大多数的状态转移对可能始终无法获得有效的回报值。这样便导致算法重复了很多盲目的训练,既浪费了时间也得不到令人满意的效果。

2.3 演化 Q 表的初始化方式

在 Q-Learning 算法中,Q 表的初始化通常为

一个全0矩阵,所以在模型探索初期存在较大的盲目性而导致算法无法快速的收敛。如何有效的初始化Q表信息对算法收敛起到了强有力的推动作用,但在无模型问题上,往往只能以随机的方式来初始化Q表信息,这未必会对算法的收敛能力起到促进作用。Q表由状态和动作的映射所组成,若初始化得到一个合适的Q表,这无疑在很大程度上加快了算法的收敛速度。可以将该问题转换为一种组合优化问题,采用演化算法的方式来寻求一种最优解,最优解的输出代表最合适的Q表。将

一个Q表视作种群中的个体,通过随机初始化若干Q表来构成演化种群NP,并采用特定的评价标准保留优良个体,较好的Q表个体中的状态-动作值满足较好的组合方式。随着演化的进行,状态-动作值的最好组合方式将被找到并作为Q表的初始化信息。图3给出了个体与种群的设定形式。

这样便产生了多个并行的解空间。基于以上方式利用演化算法生成与目标函数相关的高适应度初始种群,从而可以充分探索问题的解空间,极大地提高算法前期的探索效率。

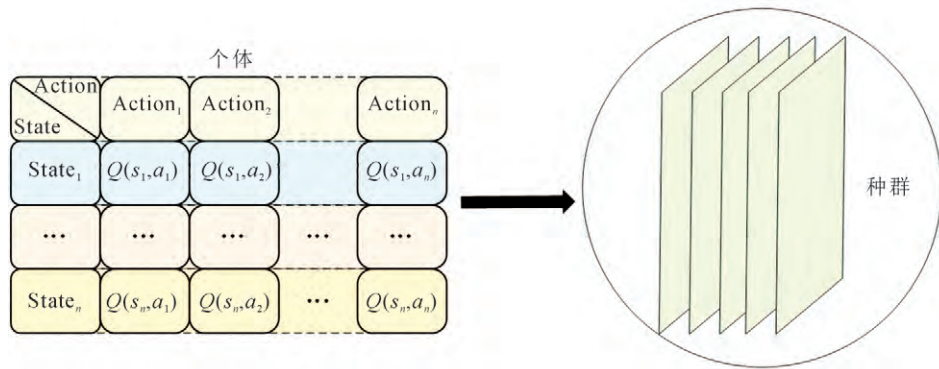


图3 个体的创建过程

Fig. 3 The process of creating an individual

2.4 DE-Q-Learning 理论思想

在求解无模型问题时,Q-Learning 采用时序差分(Temporal Difference, TD)思想来进行策略更新,通过行为值函数计算公式对当前Q表进行更新,如此反复地进行“收集-利用-更新”这一过程来找寻最优策略。对于每一步状态转移所产生Q表的更新,都将直接或间接影响后续策略的选择。由于算法在更新迭代时,对先前信息存在较大的依赖性,而在探索初期,若收集到的环境反馈信息较少,则会使算法陷入盲目勘探中,无法有效获得更加具有指导性的环境信息。

图4描绘了以DE算法为例所提出的一种新型优化算法(DE-Q-Learning)。这里从宏观层面表述了DE-Q-Learning的通用性,并重点阐明了算法的独特性。在算法框架的设计上,文中提出了一种通用的方法,使其可以自由更换演化算法类型以满足在不同问题下Q-Learning的适用性。

对比Q-Learning与DE-Q-Learning可观察到,在初始化Q表时,两者最大的区别在于同一时间所更新Q表的数量。DE-Q-Learning在空间层面出现了更加丰富的并行解集,极大程度优化了算

法的执行效率。与此同时,由于在优化过程中Q表种群信息是随机初始化的,在使用演化算法对其进行优化后,输出较优的Q表时总会伴随着有效Q值的存在。

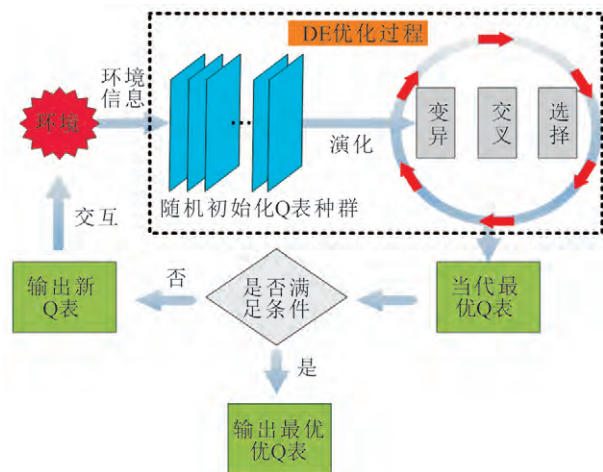


图4 DE-Q-Learning 算法流程

Fig. 4 The flow chart of DE-Q-Learning algorithm

对于如何使用DE算法来优化Q-Learning,需要重点解决表示方法、个体的定义及评价函数设定问题。由于不同的演化算法对应的演化过程不尽

相同,这里重点突出优化的思路和方式,而不针对演化算法的具体搜索过程展开对比描述。

山地车 (MountainCar) 问题是强化学习中的经典问题,如图 5 所示,小车在一条一维的轨道上,位于两座“山”之间,目标是开车上山,但是一开始小车的发动机不足以直接行驶上山,所以需要来回行驶增强动力以达到目的。接下来在 MountainCar 中进行实验来验证 DE-Q-Learning 算法的理论假设。

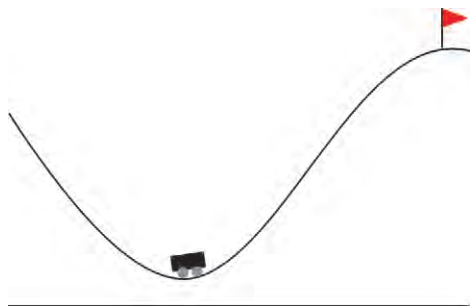


图 5 MountainCar 游戏环境

Fig. 5 The game environment of MountainCar

1) 表示方法的设定

在问题的设定上,将 Q 表作为演化种群 NP,最终将得到一个最优个体作为 Q 表的输出。在 MountainCar 游戏环境和大部分强化学习环境中,回报值的定义均为实数形式,最终计算得到的 Q 值也为实数形式,所以这里采用实数编码形式作为演化算法的个体编码以及函数值表示方法。小车的参数见表 1。

2) 演化个体的设定

在个体的定义问题上,Q-Learning 算法中的 Q 表思想上均可以视为一个状态动作对应值的二维列表。随机初始化若干 Q 表作为演化算法中的初始种群,并通过不断地演化来更新种群中的个

体信息。在山地车问题中,Q 表的内容由状态-动作对应的 Q 值组成构成,根据游戏的回报将 Q 值初始化为 $[-2, 0]$ 之间。根据表 1,小车的观测值 Observation 是 20×20 的二维表格,表示小车的速度与位置,将其作为状态,而动作集是由 3 个离散动作组成的一维表格,故整个 Q 表是状态 $\times$ 动作的三维表格。

3) 适应度函数的设定

在演化算法中,适应度函数的设定取决于优化问题的目标。而在解决强化学习问题时,针对不同的问题模型制定不同的评价函数是解决问题的最佳手段。在 MountainCar 游戏环境中,回报设置很简单:如果小车到达山顶终点的旗帜 (Position = 0.5),则奖励 0;如果小车的位置低于 0.5 (Position < 0.5),则奖励 -1。

基于以上设定,文中在 MountainCar 游戏环境中对 DE-Q-Learning 进行验证,并且使用传统 Q-Learning 以及它的改进 SA-Q-Learning 与 Double-Q-Learning 进行对比,其中 Double Q-Learning 主要思想是采用双估计器的方式来解决 Q-Learning 的过估计问题,SA-Q-Learning 则优化了算法探索和利用之间的权衡关系。算法使用的参数见表 2,超参数无具体单位,可以根据具体情况进行调整和优化,“-”代表该算法不使用此参数。

表 1 MountainCar- v_0 参数

Tab. 1 The parameters of MountainCar- v_0

状 态	小车位置/m	$[-1.2, 0.6]$
	小车速度/ m/s	$[-0.07, 0.07]$
动作集	向左加速	
	不加速	
	向右加速	

表 2 MountainCar 环境各算法实验参数

Tab. 2 Experimental parameters of each algorithm in the MountainCar environment

序 号	变量名称	Q-Learning	DE-Q-Learning	SA-Q-Learning	Double-Q-Learning
1	学习率 α	0.90	0.90	0.90	0.90
2	折扣因子 γ	0.95	0.95	0.95	0.95
3	贪婪因子 ϵ	0.50	0.50	0.50	0.50
4	种群数 NP	—	50	—	—
5	个体维度 D	—	$20 \times 20 \times 3$	—	—
6	变异因子 F	—	0.8	—	—
7	交叉因子 CR	—	$[0, 1]$	—	—
8	演化代数 G	—	20	—	—
9	初始温度	—	—	100	—
10	退火参数	—	—	0.95	—

每种强化学习算法在该游戏环境迭代 2 000 次,将每代回报取平均、最大、最小的实验结果如图 6~7 所示。根据实验结果可以看出 DE-Q-Learning 算法的累积回报在训练的各个阶段都要大于原始 Q-Learning 算法以及它的改进算法,并且收敛速度明显快于其他算法,这是由于 DE-Q-Learning 算法在训练开始前就已经获得了 DE 算法优化出的最佳 Q 表,使得智能体在与环境的交互过程中能够根据状态选择出 Q 值最大的动作,说明 DE-Q-Learning 应用于此类二维状态空间有限的环境中是有很有效的。为了达到在输出最优个体时不影响实例模型中 Q 表变化这一目的,文中提出的 DE-Q-Learning 算法在演化 Q 表时取极小运算值来初始化种群,这样既达到优化的目的,又将无关干扰因素降至最小。

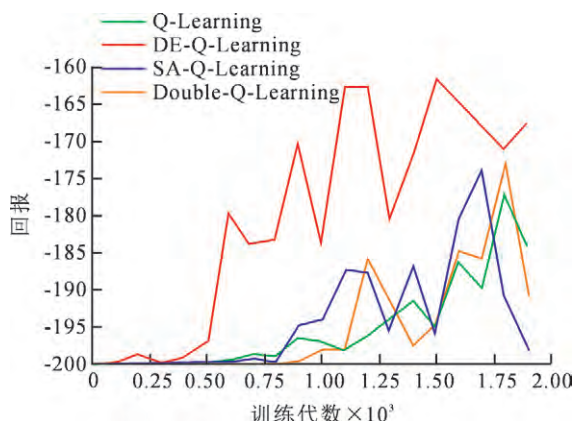


图 6 MountainCar 环境下算法的平均回报

Fig. 6 The average returns of each algorithm in the MountainCar environment

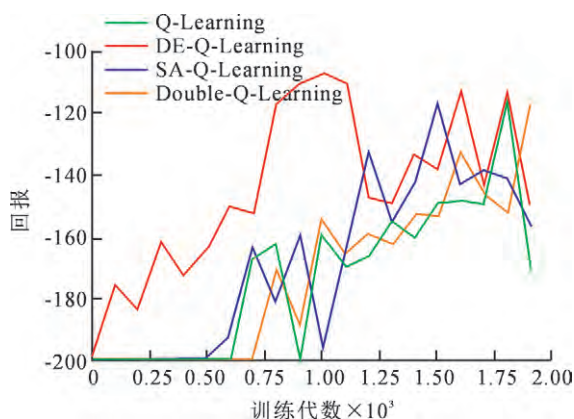


图 7 MountainCar 环境下算法的最大回报

Fig. 7 The maximum returns of each algorithm in the MountainCar environment

2.5 DE-Q-Learning 算法流程

根据 2.4 对 DE-Q-Learning 算法的理论阐述做出验证证明其是有效的,该算法的核心在于使用 DE 算法演化得到最优 Q 表,智能体能够根据 Q 表中的最优 Q 值来选取能够获得最大的收益的动作,DE-Q-Learning 算法描述如下:

Step1:初始化。针对不同问题模型构建相对应的环境信息,使用随机初始化的方式构建若干 Q 表作为种群中不同个体:

$$Q_i = \text{rand}(D, U) \quad i = 1, 2, \dots, NP, \quad (6)$$

式中: Q 为数量对应种群个数 NP , ($NP = 1, 2, \dots, n$),其编码形式采用实数编码方式; D, U 为特定环境 Q 值的最小最大值。 $\text{rand}(D, U)$ 随机生成指定范围内的 Q 值来构成 Q 表种群。

Step2:计算种群中每个个体的适应度 fitness,并找寻最优个体 Q_i 及最大适应度值 bestfitness 。

Step3:(变异)DE/rand-to-best/1/bin 为文中所采用的变异算法。

$$v_{ij}(t+1) = Q_{r1,j} + F \cdot (Q_{\text{best}} - Q_{r1,j}) + F \cdot (Q_{r2,j} - Q_{r3,j}), \quad (7)$$

式中: $i, r_1, r_2, r_3 \in \{1, 2, \dots, NP\}; j \in \{1, 2, \dots, D\}$,且 $r_1 \neq r_2 \neq r_3$; $Q_{r1,j}, Q_{r2,j}, Q_{r3,j}$ 表示从 Q 表种群中随机选取的 Q 表个体; Q_{best} 代表当代最优个体。通过以上步骤得到一个变异后的种群 v_{ij} 。

Step4:(交叉)采用二项式交叉的方式得到新的种群 u_{ij} ,令 A 为条件“ $\text{rand}(j) \leq CR$ ”, B 为条件“ $j = \text{random}(i)$ ”,则交叉个体为

$$u_{i,j}^{t+1} = \begin{cases} v_{i,j}^t, & \text{if } A \cup B \\ Q_{i,j}, & \text{otherwise} \end{cases} \quad (8)$$

Step5:(选择)计算交叉后种群 u 的适应度值 $ufitness$,并与初始种群的适应度值 $fitness$ 进行比较从而挑选出优良个体作为下一代的初始种群。

Step6:判断是否达到所需精度或最大迭代次数,若满足,则终止循环并输出最优个体。否则,返回步骤 2。

Step7:将最优个体作为 Q-Learning 的初始模型输入,通过训练找到最优策略。

DE-Q-Learning 的伪代码如算法 2 所示。

算法 2 DE-Q-Learning 伪代码

Q 表种群的随机初始化

$Q_{\text{best}} = \text{DE}(q_table_population)$ # 最优 Q 表由 DE 算法演化

输入: 环境 E , 状态空间 S , 动作空间 A , 折扣因子 γ

初始化行为值函数 $Q(s', a') = 0, \pi(a|s) = \frac{1}{|A|}$;

For $k=0, 1, \dots, m$ do # agent 的每条完整路径
初始化状态 s

For $t=0, 1, 2, 3, \dots$, do # agent 的每个训练步

通过贪心策略 ϵ of π 在环境 E 中采取动作 A

$r, s' = \text{Step}(A)$ # 执行下一个动作 A 产生奖励 r 和下一个状态 s'

$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ # 更新 Q 表

$s \leftarrow s'$

end for s is terminated

end for

$\pi^*(s) = \arg\max_a Q(s, a)$ # 根据 Q 表更新策略

3 实验与分析

为了进一步验证文中提出 DE-Q-Learning 的性能, 文中采用两种不同的二维迷宫环境进行实验, 这类游戏有两个特点: ① 动作维数小, 状态维数大。② 具有明显胜利和失败的条件, 且奖惩区别明显, 适合验证 Q-Learning 改进算法的通用性和泛化能力。

3.1 复杂迷宫环境

3.1.1 实验环境

在如图 8 所示的复杂迷宫环境中, 分别对三个算法的收敛性能进行对比实验。具体实验仿真环境为: 16 GB RAM, 512 G 硬盘, 2.60 GHz 64 位处理器, Windows 10 操作系统, 使用 Matlab2018b 仿真软件。

3.1.2 环境状态设定

为了凸显出算法在稀疏回报状况下的高效性, 该实验环境设定如下。设定黑色区域为墙体, 代表不可达状态, 模型左下角绿色区域为迷宫起点, 即智能体的初始位置。模型右上角对应的蓝色区域则为迷宫出口, 代表终止状态。模型中共有 376 个可抵达的状态, 智能体在除终止状态的其余位置上回报值 R 均为 0, 在抵达终止状态时, 回报 R 设定为 100。之所以将模型的反馈情况设置为稀疏回报, 其目的是为了突显出 DE-Q-Learning 算法的

高效性。当积极回报仅存在于终止状态时, 只有通过智能体反复地探索直至抵达目标状态从而不断地更新 Q 表, 才能促使算法收敛。这一过程起初可能是盲目的, 智能体在毫无先验信息的基础上进行探索无疑增加了算法的运行时间。

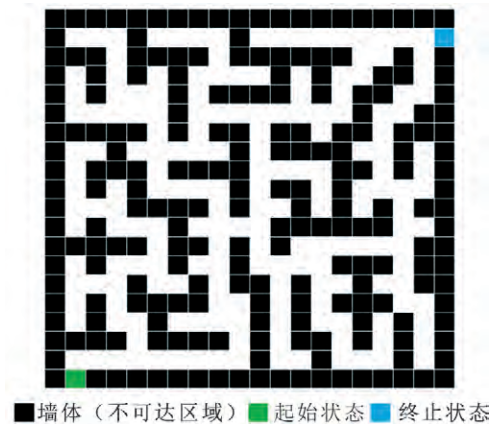


图 8 实验迷宫环境

Fig. 8 The experimental labyrinth environment

3.1.3 适应度函数以及评价指标

由于需要更加直观的评价标准即找到迷宫出口的次数, 若在规定的步长下, 智能体到达终止状态的次数较多, 则说明此时的 Q 表具备较好的环境信息。所以针对文中实验所涉及的问题, 将适应度函数设定为在指定步长下目标状态命中次数, 即

$$f(x) = \begin{cases} \text{fit_num}_i^t = \text{fit_num}_i^t + 1 & \text{if (arrive goal state)} \\ \text{fit_num}_i^t = \text{fit_num}_i^t & \text{otherwise} \end{cases}, \quad (9)$$

式中: $i \in \{1, 2, \dots, NP\}$; fit_num_i^t 为第 t 代的第 i 个个体的适应度值。对于适应度值较高的个体, 将其保留作为下一代的初始种群。另外, 用算法平均命中率作为算法优劣的评价指标, 算法在指定的步数 $Step$ 内智能体对环境进行探索, 判断是否抵达目标状态, 若抵达, 则命中数加 1, 否则, 重新开始循环。其表达式为

$$f(\text{num}) = \begin{cases} \text{num} = \text{num} + 1 & \text{if (arrive goal state)} \\ \text{num} = \text{num} & \text{otherwise} \end{cases}, \quad (10)$$

$$G(x) = \frac{\sum_{i=1}^t f(\text{num})}{t} \quad t \in \{1, 2, \dots, n\}, \quad (11)$$

式中: t 为算法运行代数; num 为智能体在 t 代时累计目标状态命中次数。通过计算函数表达 $G(x)$ 求

得算法的平均命中率。

DE-Q-Learning 算法与对比算法的具体参数见表 3。对于其它在实验中所涉及到的参数设定,这里给出统一规定。设定参数“Step”代表各算法在每次探索中所需最大步长,用来限制智能体的移动次数;设定参数“Gen”作为各算法的循环次数,代表最大训练程度。在智能体探索环境时,Q 表的每一代变化都为下一代智能体在探索环境时提供了更新信息。

3.1.4 实验结果及分析

图 9 对比了 DE-Q-Learning 与 Q-Learning、Double Q-Learning 及 SA-Q-Learning 算法在目标

状态命中率上的比较情况。横坐标表示算法的运行代数 Gen,纵坐标为命中率,这里将 Gen 的取值设为 500。图 9(a)中,DE-Q-Learning 算法在 DE 处理上的参数设置为:种群数量 NP 设置为 20,演化代数 G 设置为 20。图 9(b)中,种群数量 NP 设置为 20,演化代数 G 设置为 50。由实验结果发现,随着演化代数的提升,DE-Q-Learning 在命中率上有明显的提升,相较于 SA-Q-Learning 和 Q-Learning,其效果有着明显的提升。而 Double Q-Learning 在动作选择上采用双估计器的方式来解决过拟合问题,在前期的收敛能力上可能不及 Q-Learning 和 SA-Q-Learning。

表 3 DE-Q-Learning 与对比算法参数
Tab. 3 Parameters of DE-Q-Learning and other algorithms

序号	变量名称	Q-Learning	DE-Q-Learning		SA-Q-Learning	Double-Q-Learning
			Complex maze	Pacman		
1	学习率 α	0.90	0.90	0.90	0.90	0.90
2	折扣因子 γ	0.95	0.95	0.95	0.95	0.95
3	贪婪因子 ϵ	0.50	0.50	0.50	0.50	0.50
4	种群数 NP	—	50	50	—	—
5	个体维度 D	—	376	200×5	—	—
6	变异因子 F	—	0.8	0.8	—	—
7	交叉因子 CR	—	[0,1]	[0,1]	—	—
8	演化代数 G	—	[20,100]	20	—	—
9	初始温度	—	—	—	100	—
10	退火参数	—	—	—	0.95	—

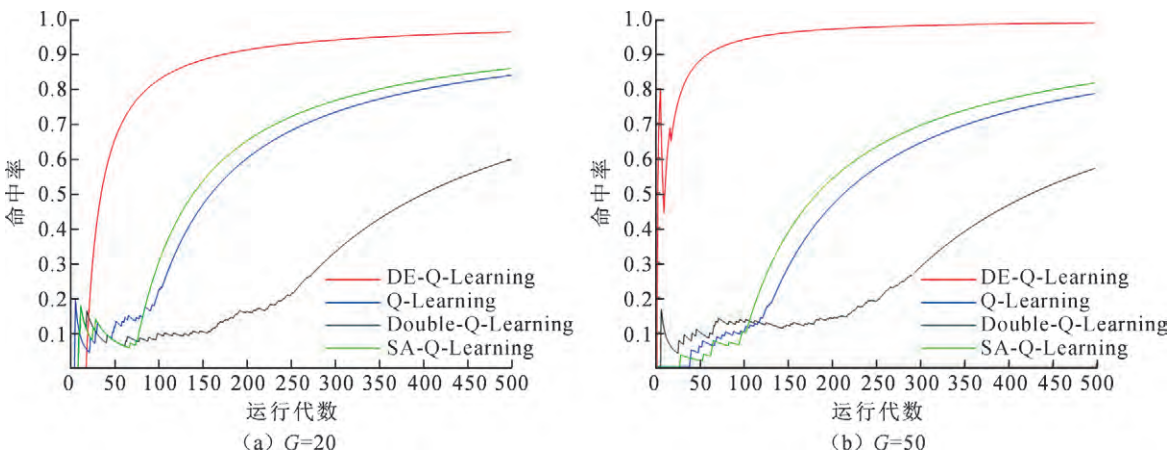


图 9 DE-Q-Learning 在 NP 为 20 时与不同算法命中率比较

Fig. 9 Comparison in the hit rate between DE-Q-Learning and other algorithms at NP of 20

图 10 对比了 Q-Learning、Double Q-Learning、SA-Q-Learning 与 DE-Q-Learning 在种群个体为 50 的情况下,分别演化 20 代与 50 代的命中

率情况。由实验结果可以看出,随着 NP 数量的增加,算法在空间上提升了搜索效率,在对模型进一步拟合后,得到误差相对较小的最优样本输出,有

效地避免了算法进行盲目搜索,更进一步地提高了算法的命中率。

图 11 对比了 DE-Q-Learning 在 NP 和 Gen 都为 100 代的情况下所得到的命中率曲线。可以观察到,伴随着演化程度的提升,算法在收敛性能上

几乎可以做到“零失误”,DE-Q-Learning 在效果上远超 Q-Learning、Double Q-Learning 及 SA-Q-Learning。

图 12 对比了 DE-Q-Learning 在 NP 和 G 不同取值下的命中率情况。

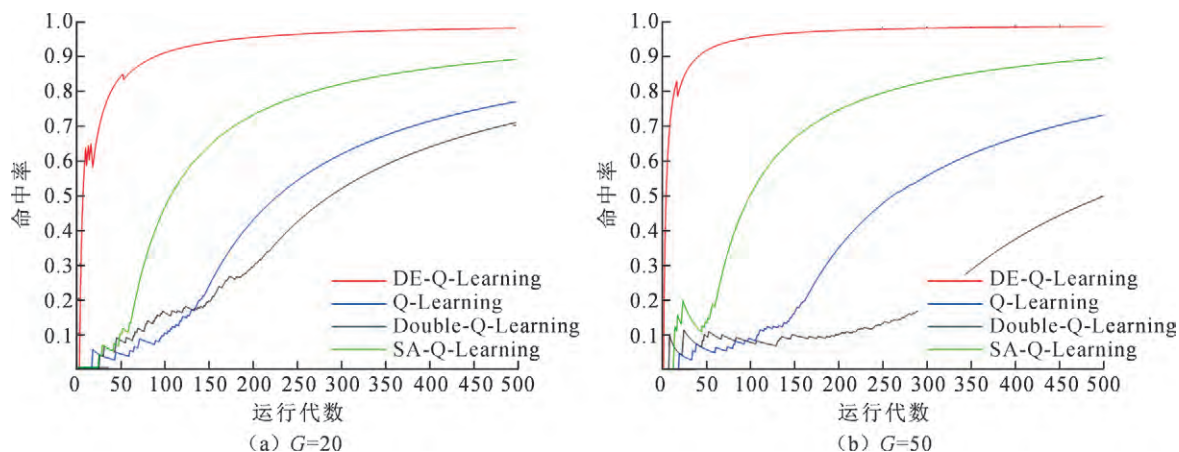


图 10 DE-Q-Learning 在 NP 为 50 时与不同算法命中率比较

Fig. 10 Comparison in the hit rate between DE-Q-Learning and other algorithms at NP of 50

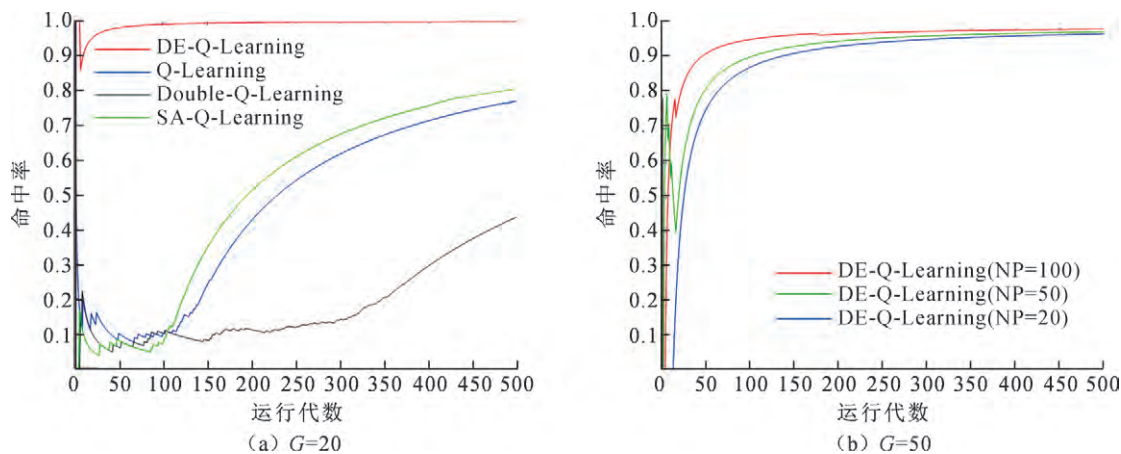


图 11 DE-Q-Learning 在 NP 为 100 时与不同算法命中率比较

Fig. 11 Comparison in the hit rate between DE-Q-Learning and other algorithms at NP of 100

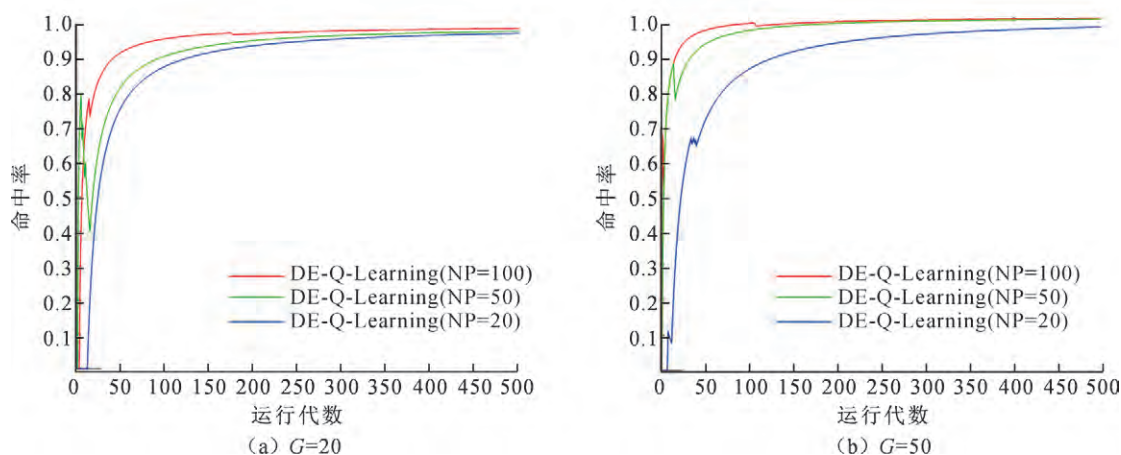


图 12 DE-Q-Learning 在 NP 和 G 不同取值下的命中率比较

Fig. 12 Comparison of the hit rates of DE-Q-Learning when NP and G have different values

图 12(a)绘出了演化种群 NP 取值为 20,演化代数分别为 20,50,100 时算法的命中率曲线,图 12(b)绘制了将演化种群 NP 提升至 50 的命中率曲线。通过实验数据可以看出,在 NP 不变的情况下,随着演化代数的增加,对算法性能有着明显的提升。而随着种群数量 NP 的提升,对算法性能也有着一定的改善,但是同时也会带来一定的代价即算法运行时间增加。

从收敛速度上,由于 DE-Q-Learning 具有独特的隐并行特点,无论从累计回报还是平均回报来看,其算法都具有优异的效果。只有采用时间作为评价标准时,才能满足对比的公平性。在“目标状态命中率”评价标准的基础上,表 4 分别记录了各算法训练 500 代所需运行时间。图 13 为最终的训练效果图,展示了改进算法在路径搜索中所训练出的最佳路径。

表 4 各算法运行 500 代所消耗时长
Tab. 4 Time consumed for each algorithm to run for 500 generations

算 法	时长/s
DE-Q-Learning	101.329 0
Q-Learning	108.591 0
Double Q-Learning	219.981 0
SA-Q-Learning	78.569 0

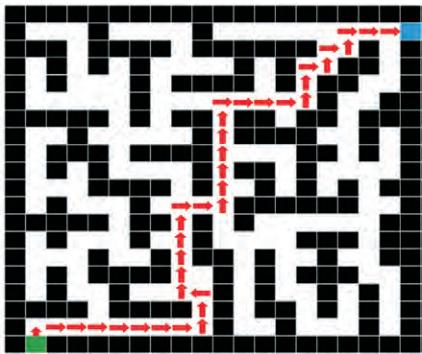


图 13 训练效果图

Fig. 13 The training result

3.2 Pacman 游戏环境

3.2.1 实验环境参数

Pacman 吃豆人游戏是上世纪 80 年代日本南梦宫游戏公司推出的一款街机游戏,后来被加州大学伯克利分校引入到强化学习的实验环境中作为实操项目,该环境在验证算法的路径搜索能力的基础上添加了动态避障的训练。文中使用该

项目中的小型实验环境 smallGrid 和中型实验环境 MediumClassic,如图 16~17 所示。文中在 16GB RAM,512 G 硬盘,2.60 GHz 64 位处理器,Windows 10 操作系统,Python3.7 的环境下进行实验。

3.2.2 环境状态设定

在 Pacman 游戏环境中,强化学习训练的智能体是如图 14~15 所示的黄色扇形移动体,白色小圆点代表小奖励,白色圆点代表大奖励,具有两只眼睛的椭圆形代表 ghost,蓝色边框代表不可穿越的墙体。智能体具有上下左右四个动作维度,状态使用 Pacman 项目中的 layout 地图样式表示,可以凸显游戏中各元素的具体位置,pacman 碰撞白色小圆点、大圆点分别获取奖励 10 分和 20 分,若与 ghost 碰撞,则会对 pacman 扣除 50 分并且游戏结束,并且在游戏中为了避免 pacman 长时间躲避 ghost 而不去探索奖励,定义 TIME_PENALTY=1 作为时间惩罚,即每秒钟 pacman 会受到-1 分的惩罚。训练的目标是智能体 pacman 通过墙体的保护上下左右移动躲避 ghost 并且尽可能吃掉所有白色圆点,以期望获得最大的收益。

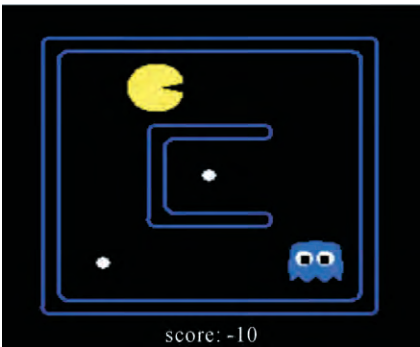


图 14 Pacman-SmallGrid 环境

Fig. 14 The environment of Pacman-SmallGrid

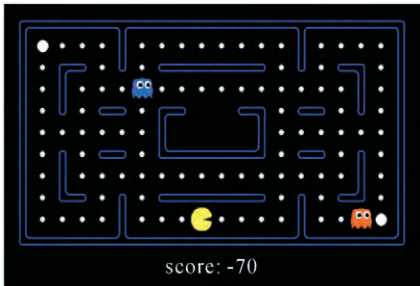


图 15 Pacman-mediumClassic 环境

Fig. 15 The environment of Pacman-mediumClassic

3.2.3 适应度函数以及评价指标

Pacman 游戏环境利用每回合智能体所得到的回报来判断训练算法的优劣,且智能体的回报根据环境返回观测数据 observation 得到,全局奖励 R 随着时间变化如式(12)所示,其中 r 代表当前状态获得的即时奖励; tp 代表时间惩罚值,取值为当前所耗费时间(s)。在 DE-Q-Learning 算法中 Q 表个体的适应度值由智能体使用该 Q 表与环境交互 n 回合的 R 取平均得到,其适应度函数为

$$R = R + r - tp, \quad (12)$$

$$f(x) = \frac{\sum_{i=1}^n R}{n}. \quad (13)$$

为了使对比实验结果更加直观,采用每一回合训练得到的 R 作为评价算法优劣的指标,DE-Q-Learning,算法与对比算法的具体参数见表 3。

3.2.4 实验结果及分析

图 16 展示了 DE-Q-Learning 算法与对比算法在 smallGrid 环境中训练 2 000 代的回报情况,可以观察到文中提出 DE-Q-Learning 算法在整个训练过程中所得到的回报是高于其他算法的,为了更加直观展现 DE-Q-Learning 算法的优越性。

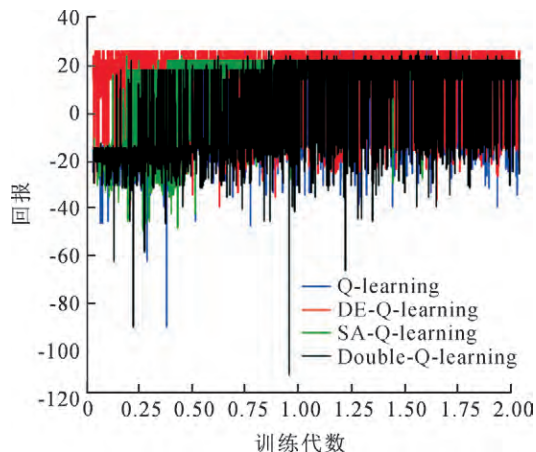


图 16 SmallGrid 环境各算法训练 2 000 代回报

Fig. 16 The return obtained after each algorithm was trained for 2 000 generations in the smallGrid environment

图 17 展示了图 16 中纵轴的累计回报 sum_reward,可以看出与其他算法不同,DE-Q-Learning 在训练前期回报远远大于其他算法,并且后期能够快速收敛。在收敛的速度上,表 5 展示了各算法训练 2 000 代所消耗的时间,其中 Double-Q-Learning 算法收敛速度最慢,SA-Q-Learning 算法收敛速度最快,其效果也好于原始 Q-Learning,DE

-Q-Learning 收敛速度仅次于 SA-Q-Learning。

式(14)为收敛速度的计算方式,其中 Ir (Improvement rate)表示改进算法相对于原始算法速度的提升率, t_o 、 t_i 分别表示原始算法与改进算法训练所耗时长。由表 5 中各算法训练 2 000 代的时间数据可知 DE-Q-Learning 相对于 Q-Learning 速度提升 26.18%,相对于 Double-Q-Learning 速度提升 42.80%,但相对于 SA-Q-Learning 其速度并无优势;而由表 6 中各算法训练 10 000 代的数据可知 DE-Q-Learning 相对于 Q-Learning, Double-Q-Learning, SA-Q-Learning 其速度分别提升了 28.58%,42.16%,15.88%。

$$Ir = \frac{(t_o - t_i)}{t_o} \times 100. \quad (14)$$

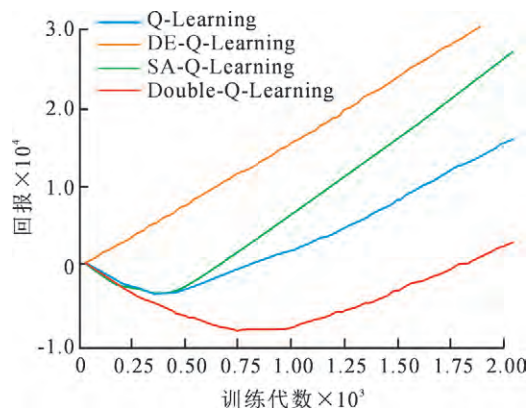


图 17 SmallGrid 环境各算法训练 2 000 代纵轴回报和
Fig. 17 The sum of the vertical axis returns after each algorithm was trained for 2 000 generations in the smallGrid environment

表 5 各算法训练 2 000 代所消耗时长

Tab. 5 Time consumed to train each algorithm for 2 000 generations

算 法	时长/s
Q-Learning	197.51
DE-Q-Learning	145.84
Double Q-Learning	254.95
SA-Q-Learning	142.61

表 6 各算法训练 10 000 代消耗时长

Tab. 6 Time consumed to train each algorithm for 10 000 generations

所消耗时长算法	时长/s
Q-Learning	7 962.81
DE-Q-Learning	5 684.63
Double Q-Learning	9 841.67
SA-Q-Learning	6 762.45

由于 mediumClassic 环境的复杂程度远远高

于 smallGrid 环境,故训练次数需要大大增加,图 18 展示了各算法在 mediumClassic 环境中训练 10 000 代的回报情况,图 19 展示了各代累计回报 sum_reward,DE-Q-Learning 算法在前期、中期、后期的回报仍然远远高于其他 Q-Learning 改进算法,表 6 展示了各算法训练 10 000 代所消耗的时间,DE-Q-Learning 算法的收敛速度快于其他改进算法,这说明 DE-Q-Learning 在复杂的环境中较其他算法更有优势。

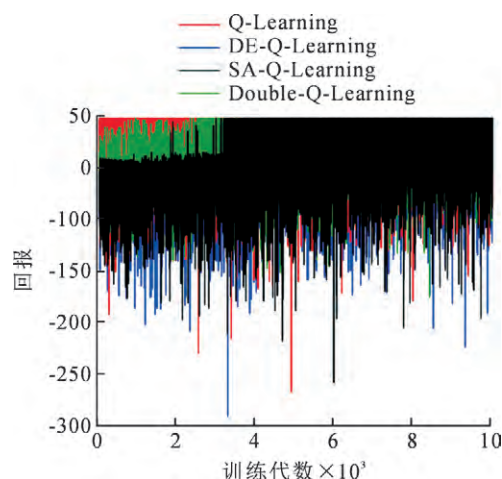


图 18 MediumGrid 环境各算法训练 10 000 代回报

Fig. 18 The return obtained after each algorithm was trained 10 000 generations in mediumGrid environment

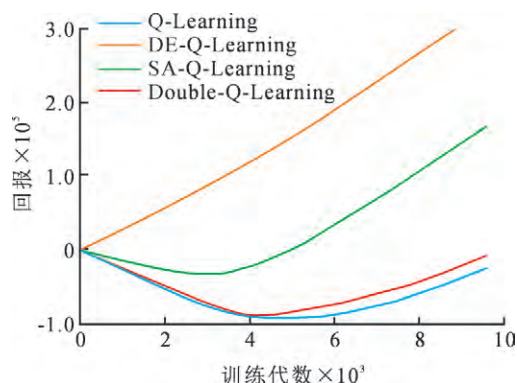


图 19 MediumGrid 环境各算法训练 10 000 代纵轴回报

Fig. 19 The vertical axis returns after each algorithm was trained for 10 000 generations in the mediumGrid environment

4 结论

为解决 Q 表规模增大时 Q-Learning 前期的盲目搜索问题,文中提出了一种使用演化算法优化 Q 表的新型算法(DE-Q-Learning)。使用 DE 算法作为演化算法实例,经过 Gym 的山地车环境验证

后,在复杂迷宫测试环境和 Pacman 游戏环境中与 Q-Learning、Double Q-Learning、SA-Q-Learning 三个算法进行比较。实验结果表明,DE-Q-Learning 无论在目标状态命中率、累计回报及收敛速度上,都具有优异的表现,算法的性能得到了明显的提升,证明该算法的有效性。从理论上讲,文中所提出的 DE-Q-Learning 可以应用于任何 Q 表有限的无模型问题,具有较好的鲁棒性,并且在演化算法的设定上,不仅局限于 DE 演化算法,其他的智能算法均可适用于该模型中。但在高维复杂环境中,由于无法创建合适的 Q 表来存储状态-动作值,且对于一些及时性策略问题 Q-Learning 算法也无法有效解决,故未来的工作将进一步研究如何利用深度强化学习优化算法解决此类高纬度、及时性的复杂问题。

参考文献:

- [1] LI H X, GONG D X, YU J J. An Obstacles Avoidance Method for Serial Manipulator based on Reinforcement Learning and Artificial Potential Field[J]. International Journal of Intelligent Robotics and Applications, 2021(5):186.
- [2] 曾明如,徐小勇,罗浩,等. 多步长蚁群算法的机器人路径规划研究[J]. 小型微型计算机系统, 2016, 37(2):366.
ZENG Mingru, XU Xiaoyong, LUO Hao, et al. Research of Robot Path Planning of Multi-step Ant Colony Algorithm[J]. Journal of Chinese Computer Systems, 2016, 37(2):366. (in Chinese)
- [3] FEINBERG E A, SHWARTZ A. Handbook of Markov Decision Processes: Methods and Applications [M]. Boston: Springer Science & Business Media, 2002.
- [4] CLIFTON J, LABER E. Q-Learning: Theory and Applications[J]. Annual Review of Statistics and Its Application, 2020, 7(1):279.
- [5] 宋丽君,周紫瑜,李云龙,等. 改进 Q-Learning 的路径规划算法研究[J/OL]. 小型微型计算机系统. <https://kns.cnki.net/kcms/detail//21.1106.tp.20230218.2208.008.html>.
SONG Lijun, ZHOU Ziyu, LI Yunlong, et al. Research on Path Planning Algorithm based on Improved Q-Learning Algorithm[J/OL]. Journal of Chinese Computer Systems. <https://kns.cnki.net/kcms/detail//21.1106.tp.20230218.2208.008.html>. (in Chinese)

- [6] 钱信,吕成伊,宋世杰. 基于优化 Q-Learning 算法的机器人路径规划[J]. 南昌大学学报(工科版),2022,44(4):396.
- QIAN Xin,LYU Chengyi,SONG Shijie. Robot Path Planning based on Optimizing Q-Learning Algorithm [J]. Journal of Nanchang University(Engineering & Technology),2022,44(4):396. (in Chinese)
- [7] MAHADEVAN S. Average Reward Reinforcement Learning: Foundations, Algorithms, and Empirical Results[J]. Machine Learning,1996,22(1/3):159.
- [8] WIERING M, SCHMIDHUBER J. Speeding up Q(λ)-Learning [C]//Machine Learning; ECML-98: 10th European Conference on Machine Learning Chemnitz, Berlin; Springer,1998:352.
- [9] HASSELT H V. Double Q-Learning[EB/OL]. (2010-12-6)[2010-12-9]. https://proceedings.neurips.cc/paper_files/paper/2010/hash/091d584fced301b44265dd8c23bfc9-Abstract.html.
- [10] AZSAR M G, MUNOS R, GHAVAMZADEH M, et al. Speedy Q-Learning[C]//Proceedings of the 24th International Conference on Neural Information Processing Systems. Granada; NIPS,2011:2411.
- [11] 郑帅,罗飞,顾春华,等. 基于双估计器的改进 Speedy Q-Learning 算法[J]. 计算机科学,2020,47(7):179.
- ZHENG Shuai, LUO Fei, GU Chunhua, et al. Improved Speedy Q-Learning Algorithm based on Double Estimator[J]. Computer Science,2020,47(7):179. (in Chinese)
- [12] GUO M Z, LIU Y, MALEC J. A New Q-Learning Algorithm based on The Metropolis Criterion[J]. IEEE Transactions on Systems, Man, and Cybernetics, Part B, Cybernetics: a publication of the IEEE Systems, Man, and Cybernetics Society,2004,34(5):2140.
- [13] 刘海涛,洪炳熔,朴松昊,等. 不确定性环境下基于进化算法的强化学习[J]. 电子学报,2006(7):1356.
- LIU Haitao, HONG Bingrong, PIAO Songhao, et al. Evolutionary Algorithm based Reinforcement Learning in The Uncertain Environments[J]. Acta Electronica Sinica,2006,34(7):1356. (in Chinese)
- [14] 焦玉民,王强,徐婷,等. 基于 GA-Q-Learning 算法的虚拟维修作业规划模型[J]. 兵工学报,2013,34(5):627.
- JIAO Yumin, WANG Qiang, XU Ting, et al. GA-Q-Learning Algorithm-Based Operation Planning Model for Virtual Maintenance Process[J]. Acta Armamentarii,2013,34(5):627. (in Chinese)
- [15] STORN R, PRICE K. Differential Evolution-A Simple and Efficient Heuristic for Global Optimization Over Continuous Spaces[J]. Journal of Global Optimization,1997,11(4):341.
- [16] JIN C, ZHU Z A, BUBECK S, et al. Is Q-Learning Provably Efficient? [J]. Advances in Neural Information Processing Systems,2018,31:4868.
- [17] ELLIOTT D L, SANTOSH K C, ANDERSON C. Gradient Boosting in Crowd Ensembles for Q-Learning Using Weight Sharing[J]. International Journal of Machine Learning and Cybernetics,2020,11:2275.
- [18] PEER O, TESSLER C, MERLIS N, et al. Ensemble Bootstrapping for Q-Learning [C]//International Conference on Machine Learning. Proceedings of the 38th International Conference on Machine Learning. Vienna; ICML,2021,139:8454.
- [19] WANG B N, GAO Y, CHEN Z Q, et al. RLGA: A Reinforcement Learning based Genetic Algorithm[J]. Acta Electronica Sinica,2006,34(5):856.
- [20] DENG W, SHANG S, CAI X, et al. An Improved Differential Evolution Algorithm and Its Application in Optimization Problem[J]. Soft Computing,2021,25:5277.

(编辑 苗静)